

MAKALAH REKAYASA PERANGKAT LUNAK
VERSION CONTROL DAN KOLABORASI TIM



Disusun oleh :

Kelompok 7

1. Muhammad Sodais Adam (2403015030)
2. Dara Ayu Aprilia (2403015057)
3. Nur Sakiya (2403015102)

Dosen Pengampu : Ir. Arry Avorizano, S.Kom., M.Kom.

FAKULTAS TEKNOLOGI INDUSTRI DAN INFORMATIKA
UNIVERSITAS MUHAMMADIYAH PROF.DR HAMKA
PROGRAM STUDI TEKNIK INFORMATIKA

KATA PENGANTAR

Puji syukur kami panjatkan ke hadirat Allah SWT karena atas rahmat dan karunia-Nya tugas Pertemuan 11 mata kuliah Rekayasa Perangkat Lunak tentang Version Control dan Kolaborasi Tim dapat diselesaikan dengan baik.

Tugas ini disusun berdasarkan materi modul perkuliahan mengenai tahapan rekayasa perangkat lunak, SDLC, analisis kebutuhan, hingga implementasi coding. Pada tugas ini penulis membuat studi kasus implementasi sistem dengan menerapkan struktur folder yang baik dan minimal dua fitur utama sesuai instruksi modul praktikum.

Penulis menyadari bahwa tugas ini masih memiliki kekurangan, oleh karena itu kritik dan saran sangat diharapkan demi penyempurnaan di masa mendatang

Jakarta, Mei 2026

DAFTAR ISI

KATA PENGANTAR-----	1
DAFTAR ISI-----	2
BAB I PENDAHULUAN-----	3
BAB II LANDASAN TEORI-----	4
BAB III PEMBAHASAN DAN STUDI KASUS-----	5
BAB IV IMPLEMENTASI SEDERHANA-----	6
BAB V KESIMPULAN DAN SARAN-----	7

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak modern, kerja sama tim menjadi salah satu faktor utama keberhasilan proyek. Semakin besar sebuah proyek, maka semakin banyak pula programmer yang terlibat dalam proses pengembangan. Kondisi ini sering menimbulkan masalah seperti konflik kode, kehilangan perubahan data, serta kesulitan dalam melacak riwayat pengembangan aplikasi.

Berdasarkan materi pada lampiran mengenai tahapan Rekayasa Perangkat Lunak (RPL), proses pengembangan perangkat lunak memerlukan dokumentasi, pengendalian perubahan, dan kolaborasi yang baik agar proyek berjalan sesuai rencana. Oleh karena itu, dibutuhkan sebuah sistem yang mampu mengelola perubahan kode secara terstruktur. Sistem tersebut dikenal sebagai Version Control System (VCS).

Git merupakan salah satu Version Control System yang paling banyak digunakan saat ini. Dengan Git, setiap anggota tim dapat bekerja pada bagian proyek masing-masing melalui branch tanpa mengganggu kode utama. Selain itu, platform seperti GitHub dan GitLab mempermudah proses kolaborasi, dokumentasi, serta penggabungan kode melalui proses commit dan merge.

1.2 Tujuan Praktikum

Tujuan dari praktikum Version Control dan Kolaborasi Tim adalah:

1. Memahami konsep dasar version control.
2. Mengetahui fungsi Git dalam pengembangan perangkat lunak.
3. Mempelajari cara membuat branch pada Git.
4. Memahami proses commit dan merge.
5. Melatih kemampuan kolaborasi dalam proyek perangkat lunak.
6. Mengurangi konflik kode antar anggota tim.

1.3 Manfaat Praktikum

Manfaat dari praktikum ini antara lain:

- Mempermudah pengelolaan perubahan kode.
- Memudahkan pelacakan histori pengembangan aplikasi.
- Membantu kerja sama tim menjadi lebih terstruktur.
- Mengurangi risiko kehilangan data.
- Mempermudah proses pengembangan sistem secara berkelanjutan.

BAB II

LANDASAN TEORI

2.1 Pengertian Version Control

Version Control adalah sistem yang digunakan untuk mencatat setiap perubahan pada file atau kode program sehingga riwayat perubahan dapat dilacak kembali. Sistem ini sangat penting dalam pengembangan perangkat lunak karena memungkinkan banyak programmer bekerja dalam satu proyek secara bersamaan.

Version control membantu tim dalam:

- Menyimpan histori perubahan kode.
- Mengembalikan kode ke versi sebelumnya.
- Mengelola perubahan dari banyak developer.
- Mengurangi konflik saat penggabungan kode.

2.2 Git

Git merupakan distributed version control system yang dikembangkan oleh Linus Torvalds. Git bekerja dengan menyimpan snapshot perubahan proyek sehingga seluruh riwayat pengembangan dapat tersimpan dengan aman.

Keunggulan Git:

1. Cepat dan ringan.
2. Mendukung kerja tim.
3. Memiliki sistem branch yang fleksibel.
4. Dapat digunakan secara offline.
5. Mendukung integrasi dengan GitHub dan GitLab.

2.3 GitHub dan GitLab

GitHub dan GitLab adalah platform berbasis cloud yang digunakan untuk menyimpan repository Git secara online.

Fungsi GitHub/GitLab:

- Menyimpan proyek secara online.
- Mempermudah kolaborasi tim.
- Memfasilitasi pull request dan merge request.
- Menyediakan dokumentasi proyek.
- Mendukung integrasi CI/CD.

2.4 Branch

Branch adalah cabang pengembangan yang digunakan untuk memisahkan pekerjaan antar anggota tim. Dengan branch, developer dapat mengembangkan fitur baru tanpa mengganggu branch utama.

Contoh branch:

- main
- development
- fitur-login
- fitur-pembayaran

2.5 Commit

Commit adalah proses menyimpan menggabungkan perubahan dari satu branch ke branch lain. Biasanya dilakukan setelah fitur selesai dikembangkan dan diuji.

BAB III

PEMBAHASAN DAN STUDI KASUS

3.1 Contoh Permasalahan

Dalam proyek pengembangan aplikasi mobile banking berbasis web dan mobile application, terdapat beberapa anggota tim yang mengerjakan fitur berbeda secara bersamaan.

Masalah yang sering terjadi:

1. File kode tertimpa oleh programmer lain.
2. Perubahan kode tidak terdokumentasi.
3. Sulit mengetahui siapa yang mengubah kode.
4. Terjadi konflik saat penggabungan fitur.
5. Kesalahan pada kode utama aplikasi.

Untuk mengatasi masalah tersebut digunakan Git dan GitHub sebagai sistem version control.

3.2 Studi Kasus

Kasus:

Tim pengembang membuat aplikasi manajemen keuangan sederhana.

Anggota tim dibagi menjadi:

Anggota	Tugas
Programmer A	Membuat fitur login
Programmer B	Membuat dashboard
Programmer C	Membuat laporan transaksi

3.3 Langkah-Langkah Penggunaan Git

1. Membuat Repository

Perintah:

`git init`

Fungsi: Membuat repository Git baru pada folder proyek.

2. Menambahkan File

Perintah:

`git add .`

Fungsi: Menambahkan seluruh file ke staging area.

3. Melakukan Commit

Perintah:

`git commit -m "Menambahkan halaman login"`

Fungsi: Menyimpan perubahan kode ke repository.

4. Membuat Branch Baru

Perintah:

`git branch fitur-login`

Fungsi: Membuat branch baru bernama fitur-login

5. Berpindah Branch

Perintah:

`git checkout fitur-login`

Fungsi: Berpindah ke branch fitur-login

6. Menggabungkan Branch (Merge)

Perintah:

`git merge fitur-login`

Fungsi: Menggabungkan branch fitur-login ke branch utama.

7. Menghubungkan ke GitHub

Perintah:

`git remote add origin https://github.com/namauser/proyek.git`

Fungsi: Menghubungkan repository lokal ke GitHub

8. Mengirim File ke GitHub

Perintah:

`git push origin main`

Fungsi: Mengirim seluruh perubahan ke repository online.

3.4 Alur Kolaborasi Tim

Berdasarkan konsep SDLC pada lampiran materi, version control mendukung tahapan pengembangan perangkat lunak secara iteratif dan terstruktur.

Alur kerja tim:

1. Project manager membuat repository
2. Setiap anggota melakukan clone repository
3. Developer membuat branch masing-masing.
4. Developer melakukan coding dan commit.
5. Perubahan dikirim ke GitHub.
6. Dilakukan review kode.
7. Branch digabungkan ke branch utama melalui merge.

3.5 Hubungan Version Control dengan SDLC

Dalam SDLC (Software Development Life Cycle), version control berperan penting pada setiap tahapan:

Tahapan SDLC	Peran Version Control
Analisis	Menyimpan dokumen kebutuhan sistem
Desain	Mengelola perubahan desain
Implementasi	Mengontrol perubahan kode program
Testing	Memudahkan rollback jika terjadi error
Maintenance	Mempermudah update dan perbaikan sistem

3.6 Keuntungan Menggunakan Git dalam Tim

1. Mempermudah koordinasi antar programmer
2. Mengurangi konflik kode.
3. Memiliki backup proyek yang aman.
4. Memudahkan dokumentasi perubahan.
5. Mendukung pengembangan aplikasi secara cepat.
6. Mempermudah rollback ketika terjadi kesalahan.

3.7 Kekurangan Version Control

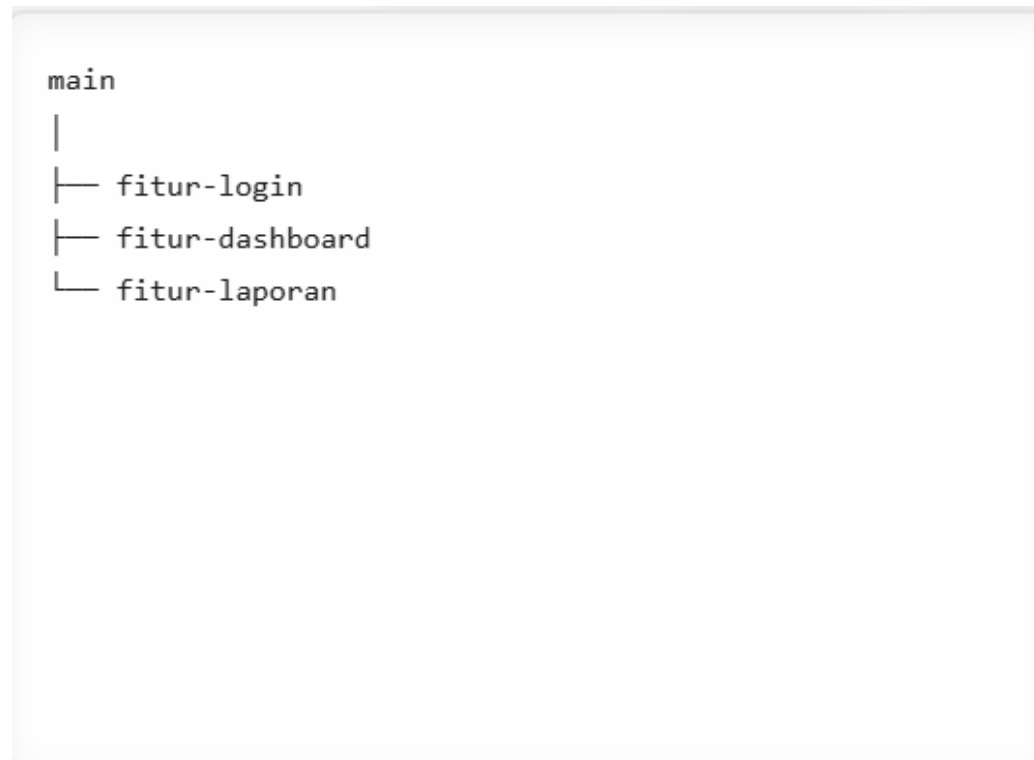
Walaupun memiliki banyak kelebihan, terdapat beberapa kekurangan:

1. Membutuhkan pemahaman command Git.
2. Konflik merge dapat terjadi jika banyak perubahan pada file yang sama.
3. Memerlukan koneksi internet untuk sinkronisasi online.

BAB IV

IMPLEMENTASI SEDERHANA

Struktur Branch Tim



Contoh Commit Message

No	Commit Message
1	Menambahkan fitur login
2	Memperbaiki validasi email
3	Menambahkan dashboard admin
4	Menambahkan laporan transaksi
5	Memperbaiki bug tampilan mobile

Simulasi Konflik dan Penyelesaian

Konflik:

Programmer A dan Programmer B mengubah file yang sama secara bersamaan.

Penyelesaian:

1. Git mendeteksi conflict.
2. Developer melakukan pengecekan manual.
3. Memilih kode yang benar.
4. Menyimpan perubahan.
5. Melakukan commit ulang

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan praktikum yang dilakukan, dapat disimpulkan bahwa Version Control System seperti Git sangat penting dalam pengembangan perangkat lunak modern. Git membantu pengelolaan perubahan kode secara terstruktur, mendukung kolaborasi tim, serta mempermudah proses pengembangan aplikasi.

Penggunaan branch, commit, dan merge membuat setiap anggota tim dapat bekerja secara terpisah tanpa mengganggu kode utama. Selain itu, GitHub dan GitLab membantu penyimpanan proyek secara online sehingga memudahkan koordinasi dan dokumentasi proyek.

Version control juga mendukung tahapan SDLC karena mampu mengontrol perubahan sistem mulai dari tahap analisis hingga maintenance.

5.2 Saran

Beberapa saran dalam penggunaan version control:

1. Gunakan commit message yang jelas dan konsisten.
2. Pisahkan fitur menggunakan branch berbeda.
3. Lakukan pull terbaru sebelum merge.
4. Gunakan GitHub/GitLab untuk backup proyek.
5. Lakukan code review sebelum penggabungan branch.
6. Tingkatkan komunikasi antar anggota tim untuk mengurangi konflik kode.

DAFTAR PUSTAKA

1. Pressman, Roger S. Software Engineering: A Practitioner's Approach. McGraw-Hill, 2019.
2. Sommerville, Ian. Software Engineering. Pearson Education, 2016.
3. Chacon, Scott & Straub, Ben. Pro Git. Apress, 2014.
4. Materi Rekayasa Perangkat Lunak – Tahapan Inisiasi dan Perencanaan Proyek.
5. Modul Praktikum RPL Pertemuan 11 – Version Control dan Kolaborasi Tim.
6. Dokumentasi Resmi Git – git-scm.com.

BAB XI

KESIMPULAN

Berdasarkan hasil implementasi perangkat lunak yang telah dilakukan, dapat disimpulkan bahwa:

1. Implementasi merupakan tahap penting dalam SLC karena mengubah desain menjadi program nyata.
2. Struktur folder yang baik membantu pengelolaan project menjadi lebih rapi.

3. Sistem manajemen tugas berhasil dibuat dengan dua fitur utama sesuai instruksi praktikum.
4. Penggunaan coding yang terstruktur mempermudah proses maintenance dan pengembangan.
5. Tahapan rekayasa perangkat lunak membantu pengembangan sistem menjadi lebih sistematis.

BAB XII

SARAN

Beberapa saran untuk pengembangan sistem:

1. Menambahkan database MySQL agar penyimpanan lebih aman
2. Menambahkan autentifikasi login pengguna
3. Membuat tampilan lebih responsif menggunakan Bootstrap

4. Menambahkan fitur edit dan hapus data
5. Mengembangkan sistem menjadi aplikasi mobile.

LAMPIRAN MATERI MODUL KULIAH

Ringkasan Materi Tahapan Rekayasa Perangkat Lunak

- 1. Inisiasi Proyek**
Tahap awal menentukan kebutuhan, stakeholder, dan tujuan sistem.
- 2. Perencanaan Proyek**
Melakukan analisis kebutuhan, estimasi biaya, jadwal dan risiko.
- 3. Analisis Sistem**

Menganalisis kebutuhan fungsional dan non-fungsional sistem.

4. Desain Sistem

Membuat rancangan database, antarmuka, dan alur sistem.

5. Implementasi/Coding

Mengubah desain menjadi kode program.

6. Pengujian

Memastikan sistem berjalan tanpa error.

7. Deployment

Sistem mulai digunakan pengguna.

8. Maintenance

Perbaikan dan pengembangan sistem setelah digunakan.

DAFTAR PUSTAKA

1. Pressman, Roger S. Software Engineering: A Practitioner's Approach. McGraw-Hill.
2. Sommerville, Ian. Software Engineering. Pearson Education.
3. Modul Praktikum Rekayasa Perangkat Lunak Pertemuan 10.
4. Materi Tahapan Rekayasa Perangkat Lunak Dosen Arry Avorizano.
5. Rosa A.S dan M. Shalahuddin. Rekayasa Perangkat Lunak Terstruktur dan Berorientasi Objek